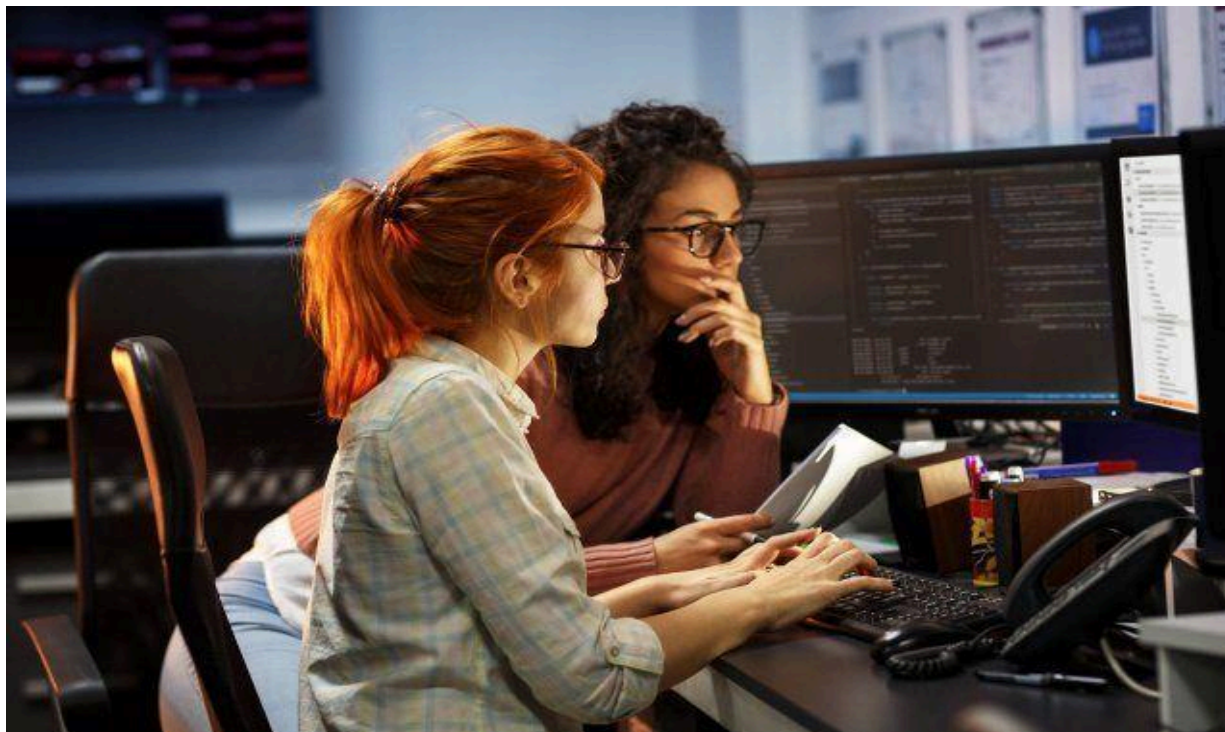




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 2

SEMANA 6

Tema: Programación y Principios de Diseño Orientado a Objetos.

Autor:

Introducción:

Bienvenidos al material de lectura **#3** de la **Unidad #2** de Programación Orientada a Objetos. En este documento estudiaremos el **encapsulamiento** en Java y su aplicación práctica mediante **clases internas y anidadas**, el uso de **genéricos** y la **organización de proyectos con paquetes** siguiendo convenciones modernas.

Este contenido busca fortalecer tu comprensión de la POO y brindarte herramientas para escribir **código más seguro, reutilizable y estructurado**, preparándote para enfrentar retos en proyectos académicos y profesionales.

Analogías del Mundo Real

Analogía 1: El cajero automático

Imagina un cajero automático. Como usuario, solo interactúas con una interfaz sencilla: introduces tu tarjeta, digitas tu PIN y seleccionas la operación que deseas realizar. No necesitas saber cómo funciona internamente el cajero: dónde están almacenados los billetes, cómo se comunica con el banco, o qué algoritmos de seguridad utiliza.

El cajero automático **encapsula** toda su complejidad interna y solo te expone una interfaz simple y segura. Si intentas abrir el cajero sin autorización, no podrás acceder a su interior.

Analogía 2: El automóvil

Cuando conduces un automóvil, utilizas el volante, los pedales y la palanca de cambios. No necesitas entender cómo funciona el motor de combustión interna, el sistema de transmisión o el diferencial. El fabricante ha **encapsulado** toda esa complejidad mecánica y te proporciona una interfaz simple para operar el vehículo.

Elementos Básicos del Encapsulamiento

Modificadores de Acceso

Los lenguajes de programación orientados a objetos proporcionan modificadores de acceso para controlar la visibilidad:

Los 4 Modificadores de Acceso

Tabla Comparativa

Modificador	Misma Clase	Mismo Paquete	Subclase (otro paquete)	Cualquier lugar	Símbolo UML
private	✓	✗	✗	✗	-
default (sin modificador)	✓	✓	✗	✗	~
protected	✓	✓	✓	✗	#
public	✓	✓	✓	✓	+

1. PRIVATE (Más Restrictivo)

Características:

- Solo accesible **dentro de la misma clase**
- Es el nivel más seguro de encapsulamiento
- **Uso principal:** Atributos de instancia

¿Cuándo usar PRIVATE?

Siempre para atributos de instancia

Métodos auxiliares que solo usa la clase internamente

Constantes que solo necesita la clase

Datos sensibles (contraseñas, tokens, información personal)

2. DEFAULT (Package-Private)

Características:

- **No se escribe ningún modificador**
- Accesible dentro del **mismo paquete**
- También llamado "package-private"
- **Uso principal:** Clases auxiliares del mismo paquete

¿Cuándo usar DEFAULT?

Clases auxiliares que solo usa tu paquete

Métodos/constructores para uso interno del paquete

Cuando quieres limitar el acceso pero permitir colaboración entre clases del mismo módulo

3. PROTECTED

Características:

- Accesible en la **misma clase, mismo paquete y subclases**
- Permite acceso desde clases hijas (herencia)
- **Uso principal:** Atributos/métodos que las subclases necesitan

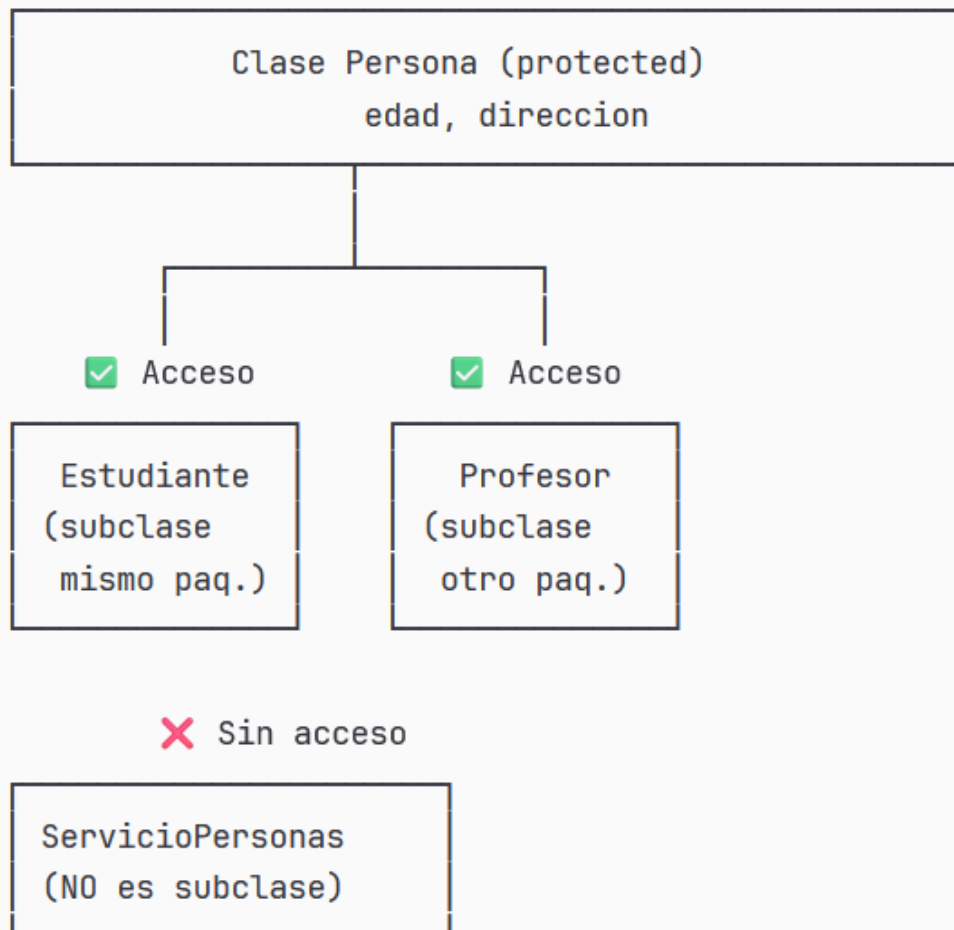


Diagrama de Acceso PROTECTED:

¿Cuándo usar PROTECTED?

Atributos que las subclases necesitan modificar
Métodos "hook" que las subclases pueden sobrescribir
Constantes que compartes con clases hijas
Cuando diseñas clases para ser extendidas (frameworks)

4. PUBLIC (Menos Restrictivo)

Características:

- Accesible desde **cualquier lugar**
- Sin restricciones de visibilidad
- **Uso principal:** Interfaces públicas (APIs), métodos de acceso

¿Cuándo usar PUBLIC?

Clases que forman parte de tu API
Métodos getter y setter
Métodos que otros necesitan llamar
Constantes públicas (con `static final`)
Constructores que deben ser accesibles externamente

Ejemplo Básico de Encapsulamiento

```
public class CuentaBancaria {  
    public String getNumeroCuenta() {  
        return numeroCuenta;  
    }  
  
    // Métodos de negocio encapsulados  
    public boolean depositar(double cantidad) {  
        if (cantidad > 0) {  
            saldo += cantidad;  
            return true;  
        }  
        System.out.println("Error: Cantidad inválida");  
        return false;  
    }  
  
    public boolean retirar(double cantidad) {  
        if (cantidad > 0 && cantidad <= saldo) {  
            saldo -= cantidad;  
            return true;  
        }  
        System.out.println("Error: Fondos insuficientes o cantidad inválida");  
        return false;  
    }  
  
    public void transferir(CuentaBancaria destino, double cantidad) {  
        if (this.retirar(cantidad)) {  
            destino.depositar(cantidad);  
            System.out.println("Transferencia exitosa");  
        }  
    }  
}
```

2.7.1. Uso de Clases Internas y Anidadas

Las clases internas (inner classes) y anidadas (nested classes) son características avanzadas de Java que permiten definir una clase dentro de otra clase. Esta técnica es una forma poderosa de encapsulamiento que ayuda a organizar el código de manera lógica.

¿Qué son las Clases Anidadas?

Una clase anidada es simplemente una clase definida dentro de otra clase. Java distingue entre varios tipos, cada uno con características y usos específicos. Piensa en ellas como oficinas dentro de un edificio: tienen acceso especial a los recursos del edificio pero están organizadas de manera independiente.

Tipos de Clases Anidadas

1. Clases Internas (Inner Classes)

Las clases internas no estáticas tienen una relación especial con la clase externa: cada instancia de la clase interna está asociada a una instancia específica de la clase externa y puede acceder a todos sus miembros, incluso los privados.

Analogía: Es como el Consejo Estudiantil dentro de una facultad. El consejo tiene acceso completo a la información y recursos de su facultad específica.

¿Cuándo usarlas? Cuando una clase solo tiene sentido en el contexto de otra clase y necesita acceso íntimo a su estado interno.

```
public class FacultadIngenieria { 3 usages 2 franciscomorales25 *
    private String nombreFacultad = "Facultad de Ingeniería UES"; 1 usage
    private int totalEstudiantes = 5000; 1 usage

    public class ConsejoEstudiantil { 2 usages 2 franciscomorales25 *
        private String presidente; 2 usages

        public ConsejoEstudiantil(String presidente) { 1 usage new *
            this.presidente = presidente;
        }

        public void organizarEvento() { no usages new *
            System.out.println("Consejo de " + nombreFacultad);
            System.out.println("Organizando evento para " + totalEstudiantes + " estudiantes");
            System.out.println("Presidente: " + presidente);
        }
    }
}

// Uso
FacultadIngenieria facultad = new FacultadIngenieria(); 1 usage
FacultadIngenieria.ConsejoEstudiantil consejo = new ConsejoEstudiantil(
    facultad.presidente, facultad.totalEstudiantes);
consejo.organizarEvento();
```

Observa cómo el **ConsejoEstudiantil** puede acceder directamente a **nombreFacultad** y **totalEstudiantes**, que son privados en la clase externa. Esta es la ventaja principal de las clases internas: acceso completo y sin restricciones.

2. Clases Anidadas Estáticas (Static Nested Classes)

Las clases anidadas estáticas no están vinculadas a una instancia específica de la clase externa. Se comportan como clases normales que están agrupadas dentro de otra por razones de organización lógica.

Analogía: Como un documento oficial de la universidad que no necesita el contexto de una facultad específica para existir.

¿Cuándo usarlas? Para agrupar clases relacionadas lógicamente, crear utilidades auxiliares, o implementar el patrón Builder.


```
public class UniversidadUES { 1 usage  @franciscomorales25 *
    private String nombre = "Universidad de El Salvador"; no usages

    public static class ValidadorCarnet { 1 usage  @franciscomorales25 *
        public static boolean esValido(String carnet) { 1 usage  new *
            return carnet.matches(regex: "[A-Z]{2}\\d{6}");
        }

        public static int obtenerAnioIngreso(String carnet) { 1 usage  new *
            if (esValido(carnet)) {
                return 2000 + Integer.parseInt(carnet.substring(2, 4));
            }
            return -1;
        }
    }
}

// Uso sin instanciar la clase externa
String carnet = "SM190045"; 2 usages
if (UniversidadUES.ValidadorCarnet.esValido(carnet)) { new *
    int anio = UniversidadUES.ValidadorCarnet.obtenerAnioIngreso(carnet); 1 usage
    System.out.println("Ingresó en: " + anio); new *
```

Las clases anidadas estáticas son útiles cuando queremos mantener clases auxiliares cerca de donde se usan, sin necesidad de crear archivos separados. No tienen acceso a los miembros de instancia de la clase externa, solo a los estáticos.

3. Clases Locales

Las clases locales se definen dentro de un método. Su alcance está limitado a ese método específico, y solo existen durante la ejecución del mismo.

Analogía: Como formar un equipo temporal de trabajo para resolver un problema específico durante una reunión.

¿Cuándo usarlas? Cuando necesitas una clase auxiliar para un algoritmo complejo que solo se usa en un método específico.

```
public class ProcesadorNotas { no usages new *
    public void calcularEstadisticas(String materia, double[] notas) { no u
        final double NOTA_APROBACION = 6.0;

        class Calculadora { 2 usages new *
            double promedio() { 1 usage new *
                double suma = 0;
                for (double nota : notas) suma += nota;
                return suma / notas.length;
            }

            int aprobados() { 1 usage new *
                int count = 0;
                for (double nota : notas) {
                    if (nota >= NOTA_APROBACION) count++;
                }
                return count;
            }
        }

        Calculadora calc = new Calculadora();
        System.out.println(materia + " - Promedio: " + calc.promedio());
        System.out.println("Aprobados: " + calc.aprobados());
    }
}
```

Las clases locales tienen acceso a las variables del método que las contiene, siempre que sean finales o efectivamente finales (no se modifican después de su inicialización).

4. Clases Anónimas

Las clases anónimas son clases sin nombre que se definen e instancian en una sola expresión. Son útiles para implementaciones rápidas de interfaces o clases abstractas que solo se usan una vez.

Analogía: Contratar a alguien para un trabajo específico de una sola vez, sin un contrato formal.

¿Cuándo usarlas? Para implementaciones simples y únicas de interfaces, especialmente en listeners de eventos o comparadores.

```
import java.util.*;

List<String> estudiantes = Arrays.asList("José", "Ana", "Carlos", "María");

Collections.sort(estudiantes, new Comparator<String>() {
    @Override
    public int compare(String e1, String e2) {
        return Integer.compare(e1.length(), e2.length());
    }
});
```

Aunque las expresiones lambda (introducidas en Java 8) han reemplazado muchos usos de clases anónimas, estas siguen siendo útiles cuando necesitas implementar múltiples métodos o mantener estado interno.

2.7.2 Genéricos

Los genéricos son una característica fundamental de Java que permite escribir código flexible y reutilizable que funciona con diferentes tipos de datos, manteniendo la seguridad de tipos en tiempo de compilación.

El Problema que Resuelven los Genéricos

Antes de Java 5, si querías crear una clase que pudiera contener diferentes tipos de objetos, tenías que usar `Object` como tipo de datos. Esto presentaba dos problemas serios: pérdida de seguridad de tipos y necesidad de castings manuales.

```
// Forma antigua sin genéricos new *
public class CajaAntigua { 2 usages new *
    private Object contenido; 2 usages

    public void guardar(Object obj) { no usages new *
        contenido = obj;
    }

    public Object obtener() { 2 usages new *
        return contenido;
    }
}

// Uso problemático
CajaAntigua caja = new CajaAntigua(); 2 usages
caja.guardar("Carnet UES"); new *
String carnet = (String) caja.obtener(); // Cast manual necesario no usages

caja.guardar(123); new *
String error = (String) caja.obtener(); // ¡Error en tiempo de ejecución!
```

Los genéricos solucionan este problema permitiendo especificar el tipo de datos que manejará una clase, manteniendo la seguridad de tipos.

Clases Genéricas

Una clase genérica se define usando parámetros de tipo entre corchetes angulares. Por convención, se usan letras mayúsculas simples: **T** para Type, **E** para Element, **K** para Key, **V** para Value.

```
public class Caja<T> { 4 usages new *
    private T contenido; 3 usages

    public void guardar(T objeto) { no usages new *
        this.contenido = objeto;
    }

    public T obtener() { 2 usages new *
        return contenido;
    }

    public boolean estaVacia() { no usages new *
        return contenido == null;
    }
}

// Uso seguro
Caja<String> cajaCarnet = new Caja<>(); 1 usage
cajaCarnet.guardar("SM190045"); new *
String carnet = cajaCarnet.obtener(); // Sin cast, seguro

Caja<Double> cajaCUM = new Caja<>(); 1 usage
cajaCUM.guardar(8.5); new *
double cum = cajaCUM.obtener(); no usages
```

Clases con Múltiples Parámetros de Tipo

Puedes definir clases con más de un parámetro genérico, útil para estructuras como pares clave-valor.

Métodos Genéricos

Los métodos también pueden ser genéricos, independientemente de si la clase que los contiene lo es o no. Esto es útil para utilidades que trabajan con diferentes tipos.

```
public class UtilidadesUES { 1 usage 2 franciscomorales25 *  
  
    public static <T> void imprimir(T[] elementos) { no usages new *  
        System.out.println("[ ");  
        for (T elemento : elementos) {  
            System.out.print(elemento + " ");  
        }  
        System.out.println("]");  
    }  
  
    public static <T> boolean contiene(T[] array, T buscar) { 1 usage 2  
        for (T elemento : array) {  
            if (elemento.equals(buscar)) {  
                return true;  
            }  
        }  
        return false;  
    }  
}  
  
// Uso  
String[] facultades = {"Ingeniería", "Medicina", "Derecho"}; 2 usages  
Integer[] codigos = {1, 2, 3, 4, 5}; 1 usage  
  
UtilidadesUES.imprimir(facultades); new *  
UtilidadesUES.imprimir(codigos); new *  
boolean tiene = UtilidadesUES.contiene(facultades, buscar: "Medicina");
```

El compilador infiere automáticamente el tipo basándose en los argumentos, aunque puedes especificarlo explícitamente si es necesario:
`UtilidadesUES.<String>imprimir(facultades).`

Interfaces Genéricas

Las interfaces también pueden ser genéricas, lo que permite crear contratos flexibles para diferentes tipos de datos.

Este patrón es extremadamente común en frameworks modernos como Spring Data, donde defines interfaces genéricas y el framework genera automáticamente las implementaciones.

Comodines (Wildcards)

Los comodines proporcionan flexibilidad adicional al trabajar con genéricos. Hay tres tipos principales:

Comodín Sin Restricciones (?)

Representa cualquier tipo. Útil cuando solo necesitas leer elementos pero no agregar.

Comodín con Límite Superior (? extends Tipo)

Acepta el tipo especificado o cualquier subtipo. Se usa cuando quieres leer datos de forma polimórfica.

Restricciones de Tipos (Bounded Type Parameters)

Puedes limitar qué tipos son aceptables usando la palabra clave `extends`.

Limitaciones de los Genéricos

Los genéricos en Java tienen algunas restricciones importantes debido a su implementación mediante "type erasure" (borrado de tipos):

No se pueden usar tipos primitivos: Debes usar las clases wrapper (Integer en lugar de int, Double en lugar de double).

No se pueden crear arrays de tipos genéricos: `new T[10]` no está permitido. Se debe usar listas o soluciones alternativas.

No se pueden instanciar el tipo genérico: `new T()` no funciona directamente. Se necesita reflexión o factories.

A pesar de estas limitaciones, los genéricos son una herramienta poderosa que mejora significativamente la seguridad y reutilización del código.

2.7.3. Paquetes

Definición

En Java, un **paquete** es un mecanismo que permite **organizar clases e interfaces en grupos lógicos**.

Funciona como una **carpeta** dentro del proyecto y ayuda a mantener el código limpio, modular y fácil de mantener.

Piensa en un paquete como una **biblioteca**: los libros (clases) están ordenados por secciones (paquetes).

Analogía Piensa en la Universidad de El Salvador como un proyecto Java completo. El campus es el proyecto, las facultades son paquetes principales, las escuelas son subpaquetes, y las aulas individuales son clases. Esta jerarquía organiza miles de estudiantes y actividades de manera coherente.

Declaración y Uso de Paquetes

Para declarar que una clase pertenece a un paquete, usamos la palabra clave `package` como primera instrucción del archivo (excluyendo comentarios):

```
package ves.ingenieria.sistemas;

public class Estudiante {
    private String carnet;
    private String nombre;

    // Constructor y métodos...
}
```


La estructura de directorios debe reflejar exactamente la estructura del paquete:

```
src/  
└─ ues/  
    └─ ingenieria/  
        └─ sistemas/  
            └─ Estudiante.java
```

Importación de Clases

Para usar clases de otros paquetes, debemos importarlas:

Usar `*` importa todas las clases del paquete, pero no los subpaquetes. Es importante notar que esto no afecta el rendimiento ni el tamaño del archivo compilado; el compilador solo incluye las clases realmente utilizadas.

Tipos de paquetes

1. Paquetes estándar (librería de Java)

Ejemplos:

- `java.util` → estructuras de datos (ArrayList, HashMap, Scanner).
- `java.io` → manejo de archivos.
- `java.sql` → conexión a bases de datos.

2. Paquetes definidos por el usuario

- Aquellos creados para organizar las clases en un proyecto propio.

Buenas prácticas al usar paquetes

- ✓ Usar **nombres en minúscula**.
- ✓ Seguir la convención de **dominio invertido** (ej: `com.google.maps`).
- ✓ Agrupar clases con responsabilidades similares.
- ✓ Mantener una estructura clara: `modelo`, `servicio`, `controlador`, `util`.

Niveles de Acceso y Paquetes

Los paquetes juegan un papel crucial en el control de acceso. El modificador **default** (ausencia de modificador) hace que un elemento sea accesible solo dentro del mismo paquete:

Esto permite crear APIs limpias donde solo expones públicamente lo que otros paquetes necesitan, manteniendo privadas las implementaciones internas.

Ventajas de Usar Paquetes

Organización: Estructuran proyectos grandes en módulos lógicos manejables.

Prevención de conflictos: Permiten tener clases con el mismo nombre en diferentes contextos.

Control de acceso: Proporcionan un nivel adicional de encapsulamiento a través del modificador default.

Distribución: Facilitan empaquetar y distribuir bibliotecas reutilizables.

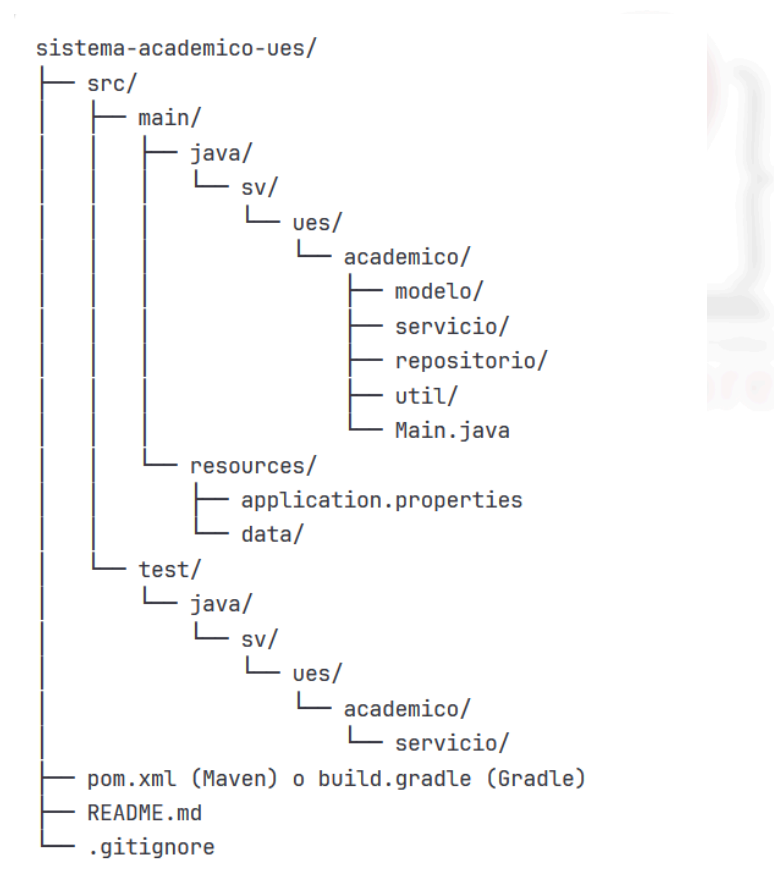
Mantenimiento: Hacen más fácil encontrar y mantener código relacionado.

2.7.3.1 Organización Estructurada de Proyectos Java Modernos

La organización profesional de proyectos Java sigue convenciones establecidas que facilitan el trabajo en equipo, la mantenibilidad y la integración con herramientas de construcción como Maven y Gradle.

Estructura Maven/Gradle Estándar

Los proyectos Java modernos siguen una estructura estándar que separa claramente el código fuente, recursos, pruebas y archivos de configuración:



Esta estructura separa claramente:

src/main/java: Código fuente principal de la aplicación **src/main/resources:** Archivos de configuración, imágenes, etc. **src/test/java:** Código de pruebas unitarias e integración **src/test/resources:** Recursos necesarios para las pruebas

Patrón de Capas

Los proyectos profesionales se organizan típicamente en capas, cada una con responsabilidades específicas:

Capa de Modelo (modelo/entities)

Contiene las clases que representan entidades del dominio. Son objetos que encapsulan datos y lógica de negocio básica.

```
// sv/ues/academico/modelo/Estudiante.java
package sv.ues.academico.modelo;

public class Estudiante { no usages &franciscomorales25 *
    private String carnet; 2 usages
    private String nombre; 3 usages
    private String apellido; 3 usages
    private double cum; 4 usages

    public Estudiante(String carnet, String nombre, String apellido) {
        this.carnet = carnet;
        this.nombre = nombre;
        this.apellido = apellido;
        this.cum = 0.0;
    }

    // Getters y setters
    public String getCarnet() { return carnet; } no usages new *
    public String getNombre() { return nombre; } no usages new *
    public String getApellido() { return apellido; } no usages new *
    public double getCum() { return cum; } no usages new *
    public void setCum(double cum) { this.cum = cum; } no usages new *

    public String getNombreCompleto() { no usages new *
        return nombre + " " + apellido;
    }

    public boolean estaAprobado() { no usages new *
        return cum >= 6.0;
    }
}
```

Las clases de modelo deben ser simples, enfocándose en representar datos y comportamientos intrínsecos de la entidad.

Capa de Repositorio (repositorio/dao)

Responsable del acceso y persistencia de datos. Abstrae los detalles de cómo se almacenan los objetos.

Esta capa aísla el resto de la aplicación de los detalles de almacenamiento. Si decides cambiar de almacenar en memoria a usar una base de datos, solo modificas esta capa.

Capa de Servicio (servicio/service)

Contiene la lógica de negocio principal. Coordina operaciones entre diferentes repositorios y aplica reglas de negocio.

```
// sv/ves/academico/servicio/ServicioEstudiante.java
package sv.ves.academico.servicio;

import sv.ves.academico.modelo.Estudiante;
import sv.ves.academico.repositorio.RepositorioEstudiante;
import java.util.List;

public class ServicioEstudiante { no usages  & franciscomorales25 *
    private RepositorioEstudiante repositorio = new RepositorioEstudiante(); 3 usages

    public boolean registrarEstudiante(String carnet, String nombre, String apellido) {
        if (repositorio.existe(carnet)) {
            System.out.println("El carnet ya existe");
            return false;
        }

        Estudiante nuevo = new Estudiante(carnet, nombre, apellido);
        repositorio.guardar(nuevo);
        System.out.println("Estudiante registrado: " + nuevo.getNombreCompleto());
        return true;
    }

    public void actualizarCUM(String carnet, double nuevoCUM) { no usages new *
        Estudiante estudiante = repositorio.buscarPorCarnet(carnet);
        if (estudiante != null && nuevoCUM >= 0 && nuevoCUM <= 10) {
            estudiante.setCum(nuevoCUM);
            System.out.println("CUM actualizado: " + nuevoCUM);
        }
    }
}
```

Buenas prácticas

Usar **dominio invertido** para los nombres de paquetes.

Separar responsabilidades en subpaquetes claros (**modelo**, **servicio**, **controlador**, **util**).

Mantener las **clases pequeñas y coherentes** con su paquete.

Evitar nombres ambiguos o genéricos como **otros/** o **varios/**.

Seguir la estructura estándar de **Maven/Gradle** para que los IDEs (IntelliJ, Eclipse, NetBeans) reconozcan el proyecto fácilmente.